```
.withTagger{
      case event: CargoAdded =>
             Set("container-tag-"+ containerId.toInt % 3, "CargoAdded")  ❶
      case _ => Set("container-tag-"+ containerId.toInt % 3)             ❷
}
```

❶ Adds these two tags when CargoAdded

❷ Adds only this tag for the rest of the events

These settings allow a maximum parallelism of 3 to read `CargoAdded` events or the rest of the events as a whole.

## *9.3.5 A peek at serialization and schema evolution*

Everything stored on disk must be serialized. Serializing the events of a persistent actor is no different than serializing the messages of an actor. The problem arises when the schema of events changes and evolves.

When Akka restores an actor from the journal, it uses the message's serializer/deserializer (SerDe) to convert the events back into objects. For example, in the container, the SerDe is the binding to `CborSerializable`. The same SerDe can be used for certain schema changes, but for others, you need to find a way to accommodate the changes.

Schema evolution occurs in most mature applications when the events that the application processes evolve and need to coexist with older versions. New features are added, bugs are found, or technical debts are addressed; events can change as new fields are added or others are lost; they can be split into multiple events or merged into one.

Let's look at this with `AddCargo(id, kind, size)`. After a while, the company realizes it wants to record the process by which the cargo is loaded into the container by a human or a robot. So, a `loadedBy` field is added to `AddCargo` and also to `CargoAdded`. Once this is done, you redeploy the new version. But events loaded from the journal have no reference to this new field.

One solution would be to migrate all data from the journal to the new event type and fill it with a third value, such as "no data" in the new `loadedBy` field. However, doing so would sacrifice the advantages of the event-sourcing approach. Events are immutable facts—the past is not there to be changed. Now imagine that instead of adding a new field, you want to get rid of one. If you delete that field in the journal, it will be unrecoverable.

With an event-sourcing approach, you can solve these problems by doing two things: choose a strategy to deserialize multiple versions of events, and decide how to load the old events into the new event type. To find SerDe for different versions of the events, you can rely on the available functions in Jackson or Google Protocol Buffers. If you follow their rules, you can safely add,